

GGUF-METADATA PREDICTION OF SINGLE-SEQUENCE LLAMA.CPP THROUGHPUT ACROSS TWO SYSTEMS

Author Name

Affiliation
author@example.com

ABSTRACT

We predict single-sequence model throughput from GGUF metadata using roofline-shaped predictors with quantization-specific scale factors fitted on reference models. The selected cohort comprises 216 phase-depth measurements on a MacBook Pro M4 Max and an NVIDIA RTX 5080. On host-specific held-out sets of four and two configurations, an active-parameter decode model obtains 13.1% and 36.1% mean absolute percentage error (MAPE), versus 49.4% and 51.9% when charging total parameters. Cross-host coefficient transfer yields 13.9% and 36.7% test MAPE, but over all target rows its error is 20.8–21.7%, versus 11.0–13.8% after target fitting. A low-bit model ladder changes ordering across host/runtime stacks. The P2 prefill baseline is unsupported on RTX, where held-out error reaches 108.2% versus 18.7% on the MacBook. GGUF structure helps on both systems, but fitted efficiencies are not universal.

Index Terms— large language models, performance prediction, llama.cpp, GGUF, mixture of experts

1. INTRODUCTION

Choosing a local language model often requires benchmarking several multi-gigabyte GGUFs. A predictor based on stored metadata would reduce candidate executions relative to a lookup table; remote header-only acquisition is outside our scope. Three effects complicate a raw memory roofline [1]: quantized kernels attain different fractions of peak bandwidth, mixture-of-experts (MoE) models store many more weights than they activate per token, and hybrid architectures allocate key-value (KV) state differently in global, sliding-window, and recurrent layers.

We ask which parts of such a predictor transfer across model families and hardware. Our first contribution is a host-adapted decode model whose target features—file size, tensor shapes, expert routing, and per-layer attention metadata—are available before inference. We evaluate a frozen family split on both unified-memory Apple Silicon and a discrete NVIDIA GPU. Second, we isolate the benefit of activated-parameter and per-layer KV accounting. Third, we report transfer limits rather than hiding them in a pooled score: one-term coefficients only partly transfer, low-bit behavior varies by system, and the tested prefill model does not replicate on RTX.

LLMCompass explores accelerator design [2]. Vidur combines profiling with serving simulation [3]. RooflineBench and LIMINAL apply hardware-aware analysis to local LLM inference [4, 5]. MoE-CAP argues that sparse utilization should be measured over activated rather than stored parameters [6]; we test the predictive effect of that accounting. Benazir and Lin characterize quantization and memory behavior on Apple Silicon [7]. Our narrower estimator targets unmeasured single-sequence GGUF families from metadata plus a small reference set, followed by an explicit cross-system falsification

test. Measurements use llama-bench from llama.cpp [8]; GGUF parsing follows its format specification [9].

2. PREDICTOR

Let measured $T_d(c)$ and $T_p(c)$ denote decode and prefill throughput. Let S be GGUF file size, P and P_a total and activated parameter counts, B_h measured bandwidth on host h , and c existing-prefix depth. We model bytes per generated token and decode tokens per second as

$$D(c) = S \frac{P_a}{P} + K(c), \quad \hat{T}_d(c) = \eta_{h,q} \frac{B_h}{D(c)}, \quad (1)$$

$$K(c) = \sum_i 2h_i d_i \min(c, w_i) b_{KV} + S_{\text{state}}. \quad (2)$$

Here q is quantization format, h_i and d_i are a layer’s KV-head count and head dimension, w_i is its window (∞ for global attention), and $b_{KV} = 2$ bytes per cached scalar for the fixed F16 cache, and S_{state} is fixed recurrent-state traffic. Recurrent layers set $h_i = 0$. Tensor dimensions and expert-routing fields estimate P_a ; if an expert bank has P_e weights, E experts, and k selected experts, then $P_a = P - P_e + (k/E)P_e$. Dense models have $P_a = P$.

For each (h, q) , $\eta_{h,q}$ is the median of $T_d D / B_h$ over training rows; an unseen format uses the host-wide median. This ratio-space fit prevents the fastest small models from dominating least squares. It is an empirical scale factor, not literal utilization: SP_a/P assumes file-wide average bytes per parameter, and both file bytes and the copy benchmark are traffic proxies, so η may exceed one. B0 is the unfitted total-size roofline $B_h/(S + K)$; B1 fits $\eta_{h,q}$ but still charges total weights; B2 uses Eqs. (1)–(2). Comparing B1 with B2 isolates active-weight accounting while holding the fitting rule fixed. Because B_h cancels after same-host fitting, it affects B0 and cross-host substitution, not target-fitted B1/B2 accuracy; F_h analogously cancels for P1/P2.

We also test the deliberately simple zero-prefix prefill model

$$\hat{T}_p = \phi_{h,q} \frac{F_h}{2P_a}, \quad (3)$$

where F_h is calibrated matrix throughput. For each depth-zero training row, we back-solve $r = T_p(2P_a)/F_h$. P1 sets ϕ_h to the median r for host h ; P2 uses the median within each host-quantization group. An unseen quantization falls back to the median of that host’s fitted group coefficients. Neither model contains an existing-prefix term, so greater depths are scope tests, not part of the fit.

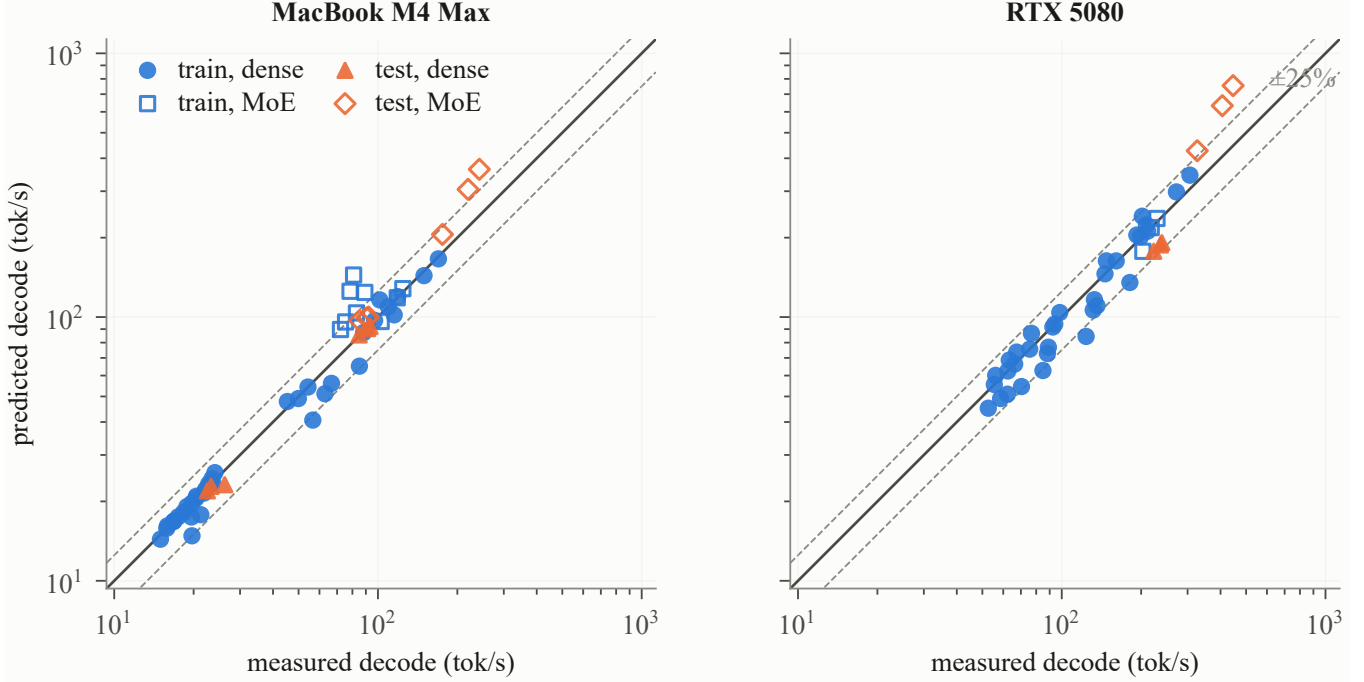


Fig. 1. B2 decode predictions, separated by host. The 99 points comprise 57 MacBook and 42 RTX observations, with 81 training and 18 held-out; dashed lines mark $\pm 25\%$. Open markers are MoE and orange markers are held-out.

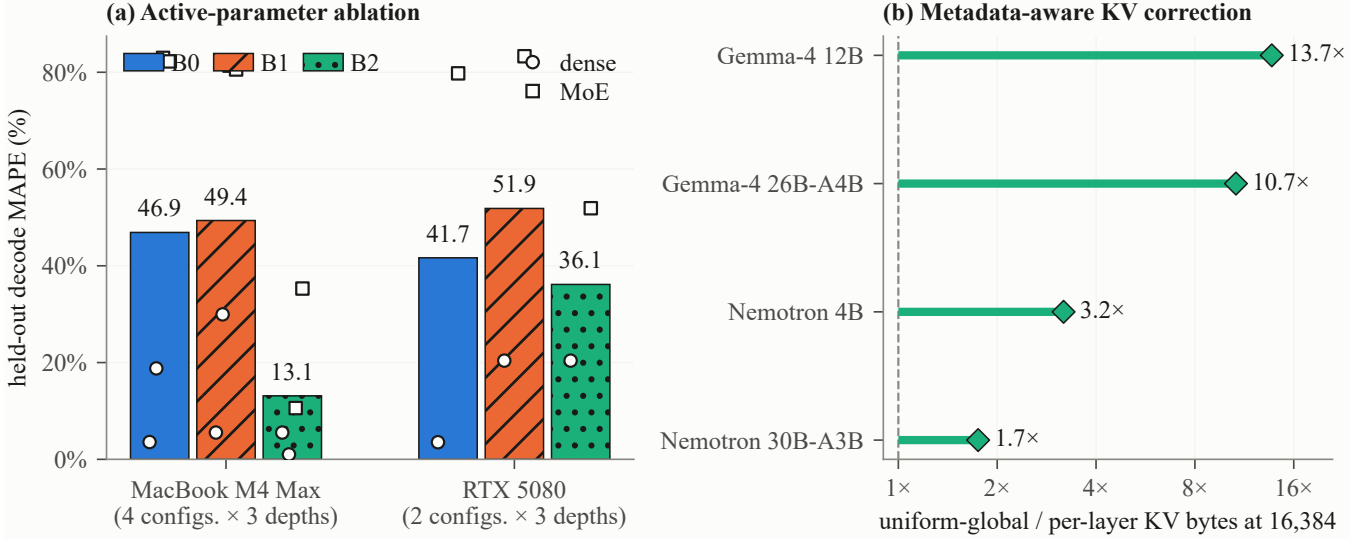


Fig. 2. Structural corrections. (a) Bars are host-specific held-out MAPE; overlaid points are per-configuration means across three depths and expose the four-versus-two test sizes. (b) From GGUF metadata, treating every layer as global overstates 16K KV bytes by as much as 13.7×

3. EXPERIMENTAL METHOD

The first host is a MacBook Pro M4 Max with 64 GB unified memory; its 12 performance cores are used as benchmark threads. Device-copy bandwidth and FP16 matrix calibration are 380.1 GB/s and 13.95 TFLOP/s. The second is an RTX 5080 with 16,302 MiB reported VRAM, CUDA, and 16 threads; device-copy and FP16 calibrations are 801.1 GB/s and 118.83 TFLOP/s. The managed RTX runner directory is labeled b10794, but its version query returned us-

age text rather than an independently verified runtime revision; the Mac revision was likewise not recovered.

The Mac manifest has 23 GGUF files; 22 files produced successful rows. After excluding three historical LLM reference probes, scoring uses 15 training and four held-out configurations from 12 base families. RTX covers 14 matching files from eight base families, scored as 12 training and two held-out configurations. Both test sets contain dense and MoE families absent from training, but

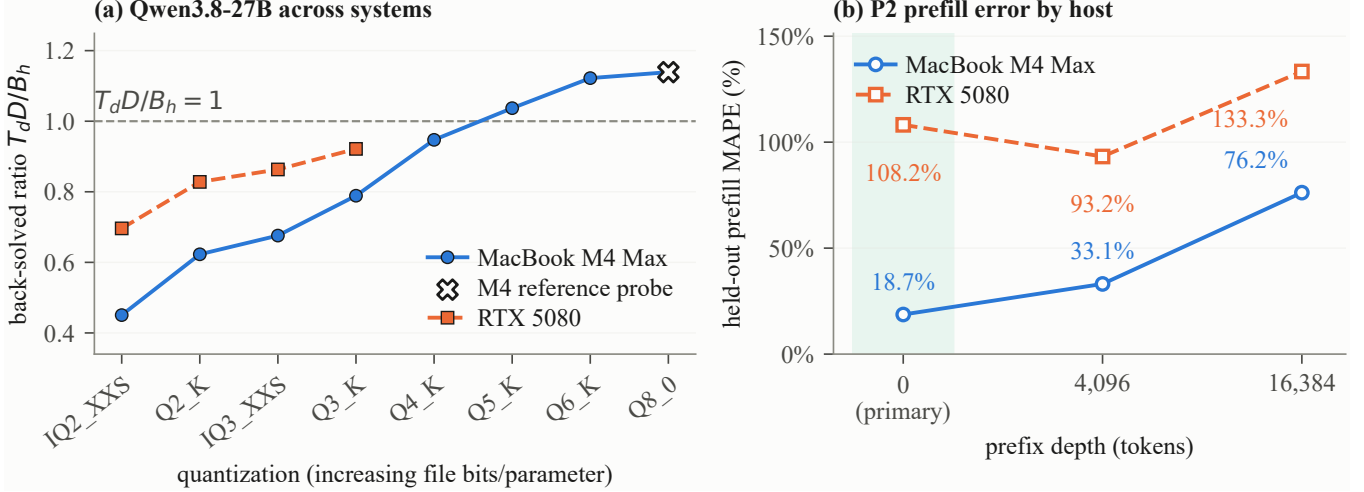


Fig. 3. Transfer diagnostics. (a) The back-solved depth-zero ratio $T_d D/B_h$ for one Qwen family varies across systems; RTX covers the first four formats. The crossed Mac Q8 point is a historical reference probe, shown for ladder context but excluded from fitting and scoring. (b) P2 prefill error is host-specific, remains poor on RTX, and rises by 16K.

only Q4 variants; RTX’s two test files are a particularly small replication. Results are therefore reported by host, never as a cohort-size-weighted headline. We retain the pre-specified Mac probe exclusion even though final exports use copy bandwidth; including those Q8 training probes leaves test B2 unchanged (supplement). Splits are fixed by base family before fitting, so all formats of a family share a label. The remaining 63.39-GB gpt-oss-120B file produced two prompt-batch failures, not a demonstrated out-of-memory failure. The supplement lists every model-card repository, exact file-name, and byte size; the evaluated repositories are cited here [10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21].

All reported throughput is single-sequence. Each configuration runs in a fresh process. At prefix depths 0, 4096, and 16384, llama-bench reports separate 512-token prompt-processing and 128-token generation microbenchmarks, not one end-to-end request. Runs use flash attention, F16 K/V, five repetitions, and 45 s settling. The requested GPU-layer count is 99. Physical RTX residency was not instrumented; Windows shared-memory spill therefore cannot be excluded for the largest files. The append-only data are reduced by an atomic selector that prefers complete protocol rows and then lower decode coefficient of variation (CV) and pre-run load, without inspecting prediction residuals.

The stability gate applies to decode. RTX’s 42 selected decode rows have 0.43% median and 1.65% maximum within-cell CV; Mac has 0.83% median and one of 66 rows at 3.04%. Prefill was not gated: 29 of 42 RTX rows exceed 3% CV and the maximum is 39.2%. We retain those observations to expose the protocol failure and give a median-sample sensitivity below. MAPE is computed over the fixed split; each decode configuration contributes its three depths.

4. RESULTS

4.1. Host-adapted decode

Figure 1 and Table 1 show the primary result. On the MacBook, B2 achieves 10.4% train and 13.1% test MAPE; on RTX, it achieves 10.1% and 36.1%. The RTX test median and maximum are 25.8% and 69.0%; its dense Nemotron 4B and MoE Ling tests average

Table 1. Host-separated MAPE (%). “Cfg.” is the number of model-format configurations; decode includes three depths per configuration, while P1/P2 are scored at depth zero.

Host	Split (cfg.)	B0	B1	B2	P1	P2
MacBook	train (15)	33.6	15.8	10.4	6.8	4.1
	test (4)	46.9	49.4	13.1	22.7	18.7
RTX 5080	train (12)	20.3	9.8	10.1	14.5	5.9
	test (2)	41.7	51.9	36.1	124.5	108.2

20.4% and 51.9%, respectively. The Mac test set has two additional files; on the exact two-file intersection, Mac MAPE is 18.2% versus RTX’s 36.1%. This remains a small replication, although B2 improves over B1 on both hosts.

Active-weight accounting substantially improves the held-out aggregate (Fig. 2a): B1’s errors of 49.4% and 51.9% fall to 13.1% and 36.1%. The effect is strongest for MoE cases: RTX’s single held-out MoE file falls from 83.3% to 51.9%, while its dense file remains at 20.4%. Capacity is controlled by stored parameters, whereas per-token weight traffic is closer to activated parameters. Separately, Eq. (2) avoids the 10.7–13.7× cache overestimates produced by a uniform-global formula for the two Gemma-4 examples (Fig. 2b). After refitting each variant, per-layer rather than uniform-global KV lowers held-out MAPE from 21.2% to 13.1% on Mac and from 37.9% to 36.1% on RTX.

4.2. Cross-host transfer is incomplete

Separate fitting should not be confused with hardware generalization. In a leave-one-host-out coefficient-transfer diagnostic, we fit B2’s median ratios on one machine and substitute the other machine’s calibrated bandwidth. Across all target rows not used to fit source coefficients, MAPE is 20.8% for 57 Mac rows and 21.7% for 42 RTX rows (medians 15.4% and 18.7%); target-fitted B2 gives 11.0% and 13.8% on those same all-row cohorts. Restricted to target test families, transfer gives 13.9% on Mac and 36.7% on RTX, close to the

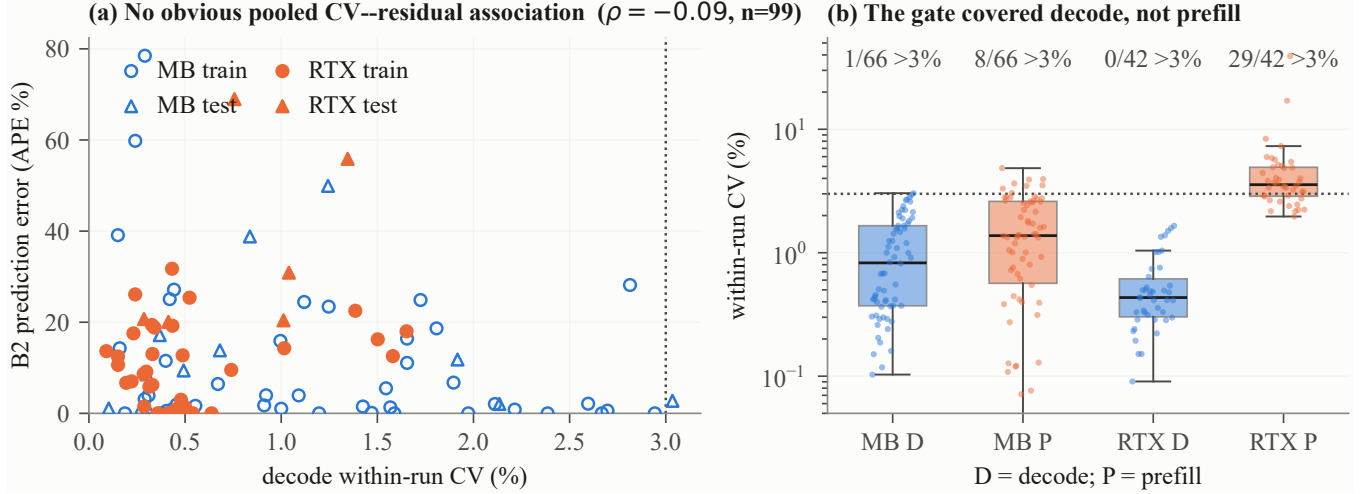


Fig. 4. Measurement diagnostics. (a) Pooled decode CV shows no obvious descriptive association with B2 error across the 99 scored rows; the dotted line is the 3% target. (b) Row-level CV distributions use all 216 selected rows, including 18 Mac reference-probe rows, and expose that the gate covered decode (D), not prefill (P).

target-fitted 13.1% and 36.1%. The shared Q4_K_M factor therefore transfers in this narrow test; the Mac Q4_K test file instead uses the source-wide median fallback. The all-row test is easier than a simultaneous model-and-host holdout because most target files also occur on the source host.

The 14 matched configurations span eight base families and weight each observed format equally; this descriptive set includes two Mac calibration-probe files excluded from prediction scoring. RTX/Mac geometric mean throughput is $2.15\times$ for zero-prefix decode, close to the $2.11\times$ copy-bandwidth ratio, and $7.21\times$ for prefill versus an $8.52\times$ compute ratio. Yet configuration-level decode ratios span $1.76\text{--}3.26\times$, ruling out one universal multiplier. Context also changes the ratios: their geometric means reach $2.30\times$ decode and $9.60\times$ prefill at 16K.

The near-family Qwen3.8-27B ladder varies across the two host/runtime stacks (Fig. 3a). IQ2 has 1.55% fewer parameters and one fewer layer. On Mac, Q2 is only 2.2% faster than IQ2 at depth zero (24.08 versus 23.57 tok/s), a gap comparable to timing variation; on RTX, IQ2 is 13.7% faster (76.76 versus 67.48 tok/s). At depth zero, across the three conventional Q4/Q8 pairs, Q4 is $1.32\text{--}1.43\times$ faster on Mac and $1.37\text{--}1.49\times$ on RTX, while its prefill rate is usually slightly lower. File compression alone therefore does not specify kernel efficiency.

4.3. Prefill is a negative result

P1/P2 reach 22.7%/18.7% held-out zero-prefix MAPE on Mac and 124.5%/108.2% on RTX (Fig. 3b); on the exact two-file intersection, Mac P2 is 12.6%. The RTX result is not solely the 39.2%-CV Ling cell: replacing every cell mean by its five-sample median and refitting still gives 85.5%. At 16K, where Eq. (3) omits prefix-dependent work, test error rises to 76.2% on Mac and 133.3% on RTX. Ten of 11 Mac and six of eight RTX quantization groups contain only one training configuration, so the per-format coefficients are weakly identified and P2’s low 4.1% and 5.9% training errors are partly resubstitution. We consequently treat prefill as unsupported under this protocol, not as a successful second regime.

4.4. Protocol and robustness checks

Decode behaves more consistently than its largest residuals. Every scored configuration slows monotonically with context. B2 held-out MAPE falls from 17.9% to 8.6% between depth zero and 16K on Mac, and from 44.5% to 25.6% on RTX. Across all 99 scored decode rows, within-run CV shows no obvious descriptive association with residual error (pooled Spearman $\rho = -0.09$; host-specific values are -0.14 and 0.16 for Mac and RTX, respectively; Fig. 4), and within-run scatter is much smaller than the largest residuals.

Correlated depth rows and the missing RTX between-run study prevent a stronger noise attribution. The supplement reports a rejected additive output-projection sensitivity analysis.

5. LIMITATIONS AND CONCLUSION

The RTX held-out set has only two Q4_K_M files and no independent between-run repeatability campaign; acquisition spans two sessions. Quantization formats often have one training family, IQ2’s Qwen file has 1.55% fewer parameters than the rest of that ladder, and execution order was not randomized. The largest RTX analytical working set exceeds reported VRAM before graph buffers, so requested full layer offload is not proof of physical residency. No GPU clock, power, or spill telemetry was recorded, and model revisions and full hashes were not frozen. These limitations preclude a universal runtime accuracy claim. All error summaries are descriptive, not population confidence estimates. The SP_a/P proxy also scales shared tensors with expert tensors and assumes balanced k/E routing; tensor-level active-byte validation was unavailable. No learning curve establishes how many reference configurations are sufficient.

Within that boundary, GGUF structure is useful: active-weight and metadata-aware KV corrections reduce held-out aggregate decode error in these cohorts, and the tested Q4_K_M factor transfers with little additional loss after target bandwidth calibration. Observed host/runtime-stack differences in low-bit behavior argue against a universal coefficient; differing runtime versions and IQ2’s slightly different architecture remain confounds. Next work should record residency and broaden the cohort.

6. REFERENCES

- [1] Samuel Williams, Andrew Waterman, and David Patterson, “Roofline: An insightful visual performance model for multi-core architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.
- [2] Hengrui Zhang, August Ning, Rohan Baskar Prabhakar, and David Wentzlaff, “LLMCompass: Enabling efficient hardware design for large language model inference,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 1080–1096.
- [3] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav Gulavani, Ramachandran Ramjee, and Alexey Tumanov, “Vidur: A large-scale simulation framework for LLM inference,” in *Proceedings of Machine Learning and Systems*, 2024, vol. 6, pp. 351–366.
- [4] Zhen Bi, Xueshu Chen, Luoyang Sun, Yuhang Yao, Qing Shen, Jungang Lou, and Cheng Deng, “RooflineBench: A benchmarking framework for on-device LLMs via roofline analysis,” *arXiv preprint arXiv:2602.11506*, 2026.
- [5] Michael Davies, Neal Crago, Karthikeyan Sankaralingam, and Christos Kozyrakis, “LIMINAL: Exploring the frontiers of LLM decode performance,” *arXiv preprint arXiv:2507.14397*, 2025.
- [6] Yinsicheng Jiang, Yao Fu, Yeqi Huang, Ping Nie, Zhan Lu, Leyang Xue, Congjie He, Man-Kit Sit, Jilong Xue, Li Dong, Ziming Miao, DaYou Du, Tairan Xu, Kai Zou, Edoardo Maria Ponti, and Luo Mai, “MoE-CAP: Benchmarking cost, accuracy and performance of sparse mixture-of-experts systems,” in *Advances in Neural Information Processing Systems*. 2025, vol. 38, pp. 90197–90223, Curran Associates, Inc., Datasets and Benchmarks Track.
- [7] Afsara Benazir and Felix Xiaozhu Lin, “Benchmarking and characterization of large language model inference on Apple Silicon,” *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 9, no. 3, pp. 1–26, Dec. 2025.
- [8] Georgi Gerganov et al., “llama.cpp: LLM inference in C/C++,” GitHub repository, <https://github.com/ggml-org/llama.cpp>, 2023.
- [9] ggml-org contributors, “GGUF file format specification,” GitHub documentation, <https://github.com/ggml-org/ggml/blob/master/docs/gguf.md>, 2026, Accessed September 2026.
- [10] Unsloth AI, “Muse-Glimmer-30B-GGUF model card,” <https://huggingface.co/unsloth/Muse-Glimmer-30B-GGUF>, 2026, Accessed September 2026.
- [11] LM Studio Community, “NVIDIA-Nemotron-3-Nano-4B-GGUF model card,” <https://huggingface.co/lmstudio-community/NVIDIA-Nemotron-3-Nano-4B-GGUF>, 2026, Accessed September 2026.
- [12] Unsloth AI, “Nemotron-3-Nano-30B-A3B-GGUF model card,” <https://huggingface.co/unsloth/Nemotron-3-Nano-30B-A3B-GGUF>, 2026, Accessed September 2026.
- [13] Unsloth AI, “Qwen3.5-4B-GGUF model card,” <https://huggingface.co/unsloth/Qwen3.5-4B-GGUF>, 2026, Accessed September 2026.
- [14] Unsloth AI, “Qwen3.5-9B-GGUF model card,” <https://huggingface.co/unsloth/Qwen3.5-9B-GGUF>, 2026, Accessed September 2026.
- [15] Unsloth AI, “Qwen3.6-35B-A3B-GGUF model card,” <https://huggingface.co/unsloth/Qwen3.6-35B-A3B-GGUF>, 2026, Accessed September 2026.
- [16] Unsloth AI, “Qwen3.8-27B-GGUF model card,” <https://huggingface.co/unsloth/Qwen3.8-27B-GGUF>, 2026, Accessed September 2026.
- [17] ggml-org contributors, “SmolLM3-3B-GGUF model card,” <https://huggingface.co/ggml-org/SmolLM3-3B-GGUF>, 2026, Accessed September 2026.
- [18] Google, “gemma-4-12B-it-qat-q4_0-gguf model card,” https://huggingface.co/google/gemma-4-12B-it-qat-q4_0-gguf, 2026, Accessed September 2026.
- [19] Unsloth AI, “gemma-4-26B-A4B-it-GGUF model card,” <https://huggingface.co/unsloth/gemma-4-26B-A4B-it-GGUF>, 2026, Accessed September 2026.
- [20] ggml-org contributors, “gpt-oss-20b-GGUF model card,” <https://huggingface.co/ggml-org/gpt-oss-20b-GGUF>, 2026, Accessed September 2026.
- [21] Bartowski, “inclusionAI/Ling-mini-2.0-GGUF model card,” https://huggingface.co/bartowski/inclusionAI_Ling-mini-2.0-GGUF, 2026, Accessed September 2026.